

SCOUT: Per-Context Reset Curricula for Sparse-Reward Reinforcement Learning

Siddharth Aphale¹, Ayushman Singh²

¹Stanford University

²Sesame AI

saphale@stanford.edu, ayushman@sesame.com

Abstract

Sparse-reward reinforcement learning often fails because rollouts from the unassisted evaluation start rarely reach later task stages. Reset curricula address this by starting some training rollouts from easier intermediate states, called scaffolds. Such a curriculum faces two decisions: *scaffold access*, obtaining informative starts, and *scaffold allocation*, deciding how quickly that assistance is removed. Most prior curricula pace removal on one shared schedule, which can fail when task instances, or contexts, learn at different rates. We introduce **SCOUT**, an online, learner-agnostic reset controller that gives every context its own curriculum. Using only binary rollout success, SCOUT removes assistance after sustained success, restores it after failure, and cautiously tests a harder start when progress stalls, without changing the reward, optimizer, or learner. A counting construction shows that synchronized global pacing can be insufficient when contexts need conflicting amounts of assisted practice. Across six navigation and manipulation settings, scaffold access improves learning and enables success in three where unassisted training fails within the reported budget. In a constructed pacing conflict, each tested global schedule leaves one group unsolved, while SCOUT solves both. Average success can conceal this failure, so we also report the least successful group. Group-level pacing works when learning differences follow known groups but can fail when they occur within one group. SCOUT needs no group labels and remains consistently strong in both cases. A reset curriculum should remove assistance at the scale where learning progress differs.

Introduction

An agent cannot learn the later stages of a task if it almost never reaches them. A reward given only for the final placement provides little guidance to a policy that cannot yet grasp the object in a long-horizon task such as picking it up and placing it on a shelf. Hindsight relabeling can reuse states that the agent visits, but it cannot create experience in stages the agent never reaches (Andrychowicz et al. 2017). Training can instead begin partway through the task in simulators that support state restoration. We study how to choose these training starts so that agents learn long-horizon, sparse-reward tasks from the unassisted starts used for evaluation.

We call a temporary, easier training start a *scaffold*. A grasped-object state, for example, lets the agent practice placement before it can reliably perform the preceding grasp. Several ordered scaffolds form a ladder from an easy state

near success back to the unassisted evaluation start, the *target start*. A *reset curriculum* chooses where on this ladder each training rollout begins and gradually removes the assistance (Florensa et al. 2017, 2018). We call the rate of removal the curriculum’s *pace*. The curriculum must solve two problems: *scaffold access*, obtaining useful intermediate starts, and *scaffold allocation*, deciding how much training each start receives as the policy improves. We call each task instance, such as an object–goal pair, a *context*.

Prior reset and start-state curricula have mainly studied access. Demonstration-based methods such as ACED and RFCL also pace progress separately along each demonstration (Dai, Hofmann, and Williams 2021; Tao et al. 2024), but they do not ask which contexts should share a pace. A curriculum can use one pace for all contexts, one per task group, or one per context; we study when each granularity is sufficient.

On-table contexts in our manipulation setting soon benefit from unassisted practice, whereas long-range in-air contexts require much more practice from grasped starts. Removing assistance early abandons the hard contexts; removing it late wastes training on the easy ones. Average success can hide this failure because the easy contexts dominate the score, so we evaluate the least successful group as well. A counting construction shows the failure is structural: some context pairs cannot both be solved by any synchronized global schedule (Proposition 1). Curriculum pacing must therefore operate at the same granularity as the differences in learning progress.

We introduce **SCOUT**, a reset controller that makes this pacing decision separately for every context. SCOUT keeps a *frontier* for each context: the current rung of its scaffold ladder. Sustained success moves the frontier toward the unassisted target start, failure restores assistance, and the controller cautiously tries a harder rung when progress remains inconclusive. SCOUT observes only whether each rollout succeeded, so it works with different RL algorithms without changing their reward or optimizer.

Our experiments separate access from allocation. Access to scaffolds improves learning across six navigation and manipulation settings and enables success in three where target-start training does not succeed within the reported budget. No allocation rule is uniformly best when contexts progress at similar rates; allocation matters when progress differs. SCOUT succeeds on both groups in a constructed manipulation conflict, while each tested global schedule

fails on one. Pooling within known task groups works when learning differences follow those groups but can abandon hard contexts when they occur inside one. SCOUT needs no group labels and remains consistently strong in both cases. Ladders built from one successful reference trajectory per context retain the benefit of hand-built scaffolds while reducing manual level design.

Contribution. This paper identifies reset curricula as a *granularity-matching allocation problem*: assistance should be paced at the scale where learning progress differs. SCOUT implements this idea with per-context frontiers that can advance, retreat, or test a harder level, and it requires no group labels. The experiments distinguish the general benefit of scaffold access from the narrower benefit of local allocation.

Related Work

Start-state, reset, and trajectory-derived curricula. Reverse Curriculum Generation expands starts outward from the goal as competence improves (Florensa et al. 2017). Goal-GAN samples goals near the learner’s frontier (Florensa et al. 2018). Related frameworks select start states by performance estimates (Wöhlke, Schmitt, and van Hoof 2020), track progress along demonstrations (Duan et al. 2025), or scale reset diversity for multiphase manipulation (Yin et al. 2026). These methods largely improve *access*. These methods use a shared global schedule or pace each demonstration separately when they pace assistance. RFCL, the closest prior method, keeps an independently paced reverse-curriculum frontier for each demonstration (Tao et al. 2024). ACED advances reset sections along each demonstration (Dai, Hofmann, and Williams 2021). Our focus is instead the unit that shares a curriculum pace, and we compare one global pace, one pace per task group, and one pace per context. SCOUT indexes its frontier by a general task context, removes assistance over an ordered reset ladder that need not represent trajectory time, and can restore assistance after failure or probe a harder level when progress stalls, using binary success without imitation or reward shaping. These differences matter most on coarse scaffold ladders, where component-matched versions of both advance rules perform worse because they cannot retreat (Results). Our main contribution is the pacing-granularity question and its evaluation.

Task distribution, teacher, and procedural curricula. Curriculum learning was originally framed as ordering examples from easy to hard (Bengio et al. 2009). In RL, automatic curriculum and teacher–student methods select tasks by learning progress, difficulty, or regret (Graves et al. 2017; Matiisen et al. 2020; Portelas et al. 2020; Narvekar et al. 2020). Self-paced RL moves the training distribution toward the target (Klink et al. 2020). Procedural methods such as Prioritized Level Replay, PAIRED, and ACCEL select or generate levels near the frontier of learnability (Jiang, Grefenstette, and Rocktäschel 2021; Dennis et al. 2020; Parker-Holder et al. 2022). DISCOVER and ACDC build curricula for sparse-reward goal reaching and manipulation (Diaz-Bone et al. 2025; Wang et al. 2026). Both teacher families fail on the hard group in both group-level pacing conflicts when tested directly on our context bank (Results).

Goal-conditioned off-policy learning and relabeling. Our continuous settings build on SAC and Hindsight Experience Replay (Haarnoja et al. 2018; Andrychowicz et al. 2017). Hindsight relabeling converts failed rollouts into useful updates by substituting achieved goals, but it can use only states the policy visits. Scaffolds and relabeling are therefore complementary. Scaffolds change the state distribution that the learner experiences, while relabeling extracts more supervision from those states. SCOUT controls the former and leaves the latter unchanged.

Method

Problem setup. Let $\mathcal{X}$ denote a finite training context bank and let $\ell \in \{0, \dots, L\}$ index scaffold levels. A reset operator $G(x, \ell)$ maps a context x and scaffold level ℓ to an initial state distribution $\rho_0^{x, \ell}$. Level $\ell = 0$ is the unassisted target distribution used for evaluation. Larger ℓ are training-only starts with domain-specific semantics.

We call each pair (x, ℓ) a context-scaffold cell. SCOUT maintains one active frontier f_x per context, initialized to the easiest level L . Training draws resets from active cells (x, f_x) . Evaluation always uses disjoint held-out contexts at $\ell = 0$ (Fig. 1).

Controller requirements. SCOUT is an online reset distribution controller: it observes binary rollout success at the active cell and selects the cell for the next training reset. This interface makes no assumptions about the optimizer. However, the controller must allocate under non-stationary training because the policy changes over time. The design is therefore *evidence-aware*: it acts only when it has enough recent evidence, and old evidence decays. It is also *transfer-directed*: it moves every context toward the unassisted target start used for evaluation.

Success statistics. SCOUT maintains an exponential moving average of binary rollout success for each active cell, $s_{x, \ell} \leftarrow (1 - \beta)s_{x, \ell} + \beta r$ with $r \in \{0, 1\}$. It also maintains an evidence count, a transition cooldown, a recency counter $c_{x, \ell}$, and a staleness counter d_x . The staleness counter records eligible controller updates since the frontier last moved. SCOUT makes a frontier decision only after collecting enough evidence and completing the cooldown. This prevents a single lucky success or failure from causing a transition.

Active cell sampling. Each controller step samples the active cells by a success-only score

$$\text{score}(x) = \lambda_h s_{x, f_x} (1 - s_{x, f_x}) + \lambda_p \frac{L - f_x}{L} + \lambda_{\text{stale}} \log(1 + c_{x, f_x}), \quad (1)$$

The first term favors cells near intermediate competence. The second, transfer-directed term mildly favors less assisted cells. The third increases coverage of cells that have not been sampled recently. The scores define a softmax distribution with a uniform exploration floor ϵ and a per-cell cap $q_{\max}$. Thus, a transient failure cannot permanently exclude a context. We keep the coefficients fixed across all settings and conditions.

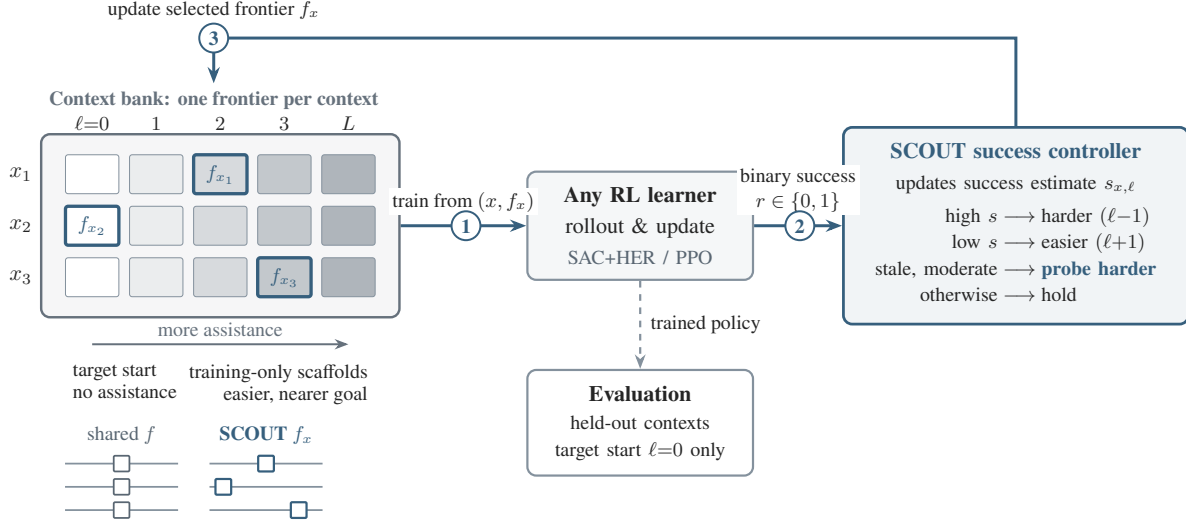


Figure 1: **SCOUT controls per-context scaffold frontiers.** Rows are contexts, columns training-only scaffold levels, with $\ell=0$ the target start used for held-out evaluation. SCOUT trains from the active cell (x, f_x) and moves each frontier using binary success (advance, retreat, or a cautious harder step).

Frontier transition. Each context updates its frontier according to the current success estimate after the evidence and cooldown conditions are satisfied:

$$f_x \leftarrow \begin{cases} \max(0, f_x - 1), & s_{x, f_x} \geq \tau_{\text{high}}, \\ \min(L, f_x + 1), & s_{x, f_x} \leq \tau_{\text{low}}, \\ \max(0, f_x - 1), & d_x \geq N_{\text{stale}} \wedge s_{x, f_x} > \tau_{\text{low}}, \\ f_x, & \text{otherwise.} \end{cases} \quad (2)$$

High success removes assistance, and low success restores it. The third case is the **antistall** transition. A success-threshold controller without this transition has a dead band $(\tau_{\text{low}}, \tau_{\text{high}})$. A context in this range is neither unsuccessful enough to retreat nor successful enough to advance, so it can remain there indefinitely. SCOUT takes one harder step after N_{stale} eligible updates without a transition. If this step is premature, the ordinary failure transition restores assistance. Antistall therefore encourages cautious progress without preventing recovery. Algorithm 1 gives the full controller loop.

Trajectory-derived scaffold construction. We also construct G automatically from one successful reference trajectory $\tau_x = (s_0, \dots, s_T)$ per training context. We define a scalar progress coordinate $p(t) \in [0, 1]$ and serialize the simulator state nearest each of five fixed progress quantiles, shared across tasks: $\{0, 0.25, 0.5, 0.75, 0.9\}$. Only the state codec differs across environments. The terminal quantile for discrete DoorKey is 1.0, so the most assisted level is adjacent to the goal. These states become the scaffold levels, with $\ell = 0$ at the target start and larger ℓ later along τ_x . This construction does not use semantic stage labels. The environment restores the selected state at reset, and training uses the original sparse reward. We validate every state offline before RL begins. The retained gain ratio R_{retain} is the fraction of the hand-built scaffold’s gain recovered by the trajectory-derived scaffold.

Algorithm 1 SCOUT curriculum controller

- 1: **Input:** context bank $\mathcal{X}$, levels $\{0, \dots, L\}$, reset operator G
thresholds $\tau_{\text{low}}, \tau_{\text{high}}$, stale limit N_{stale}
 - 2: Initialize $f_x \leftarrow L$, success averages, evidence counts, cooldowns, stale counters
 - 3: **for** each training iteration **do**
 - 4: sample context x from the active cell distribution (Eq. 1) with an exploration floor
 - 5: reset environment from $G(x, f_x)$; collect rollout; update the learner (SAC+HER / PPO)
 - 6: update success average and evidence for (x, f_x)
 - 7: **for** each context whose evidence and cooldown conditions hold **do**
 - 8: apply the frontier transition of Eq. 2
 - 9: **end for**
 - 10: **end for**
 - 11: **Evaluate only on held-out contexts at level $\ell = 0$.**
-

A value ≥ 1 means that it matches or exceeds the hand-built scaffold. This procedure constructs scaffolds *given* reference trajectories and reset access; it does not discover them from scratch.

A structural limit of synchronized pacing. A counting construction shows that synchronized global schedules can fail when contexts require different amounts of assisted practice. It isolates the scheduling conflict tested below, rather than the full reinforcement learning dynamics.

Proposition 1 (Synchronized global pacing can be insufficient). *Every $K > 1$ admits a two-context, two-level scaffold curriculum with per-context budget $K + 1$ such that a context-local schedule solves both contexts, while every synchronized*

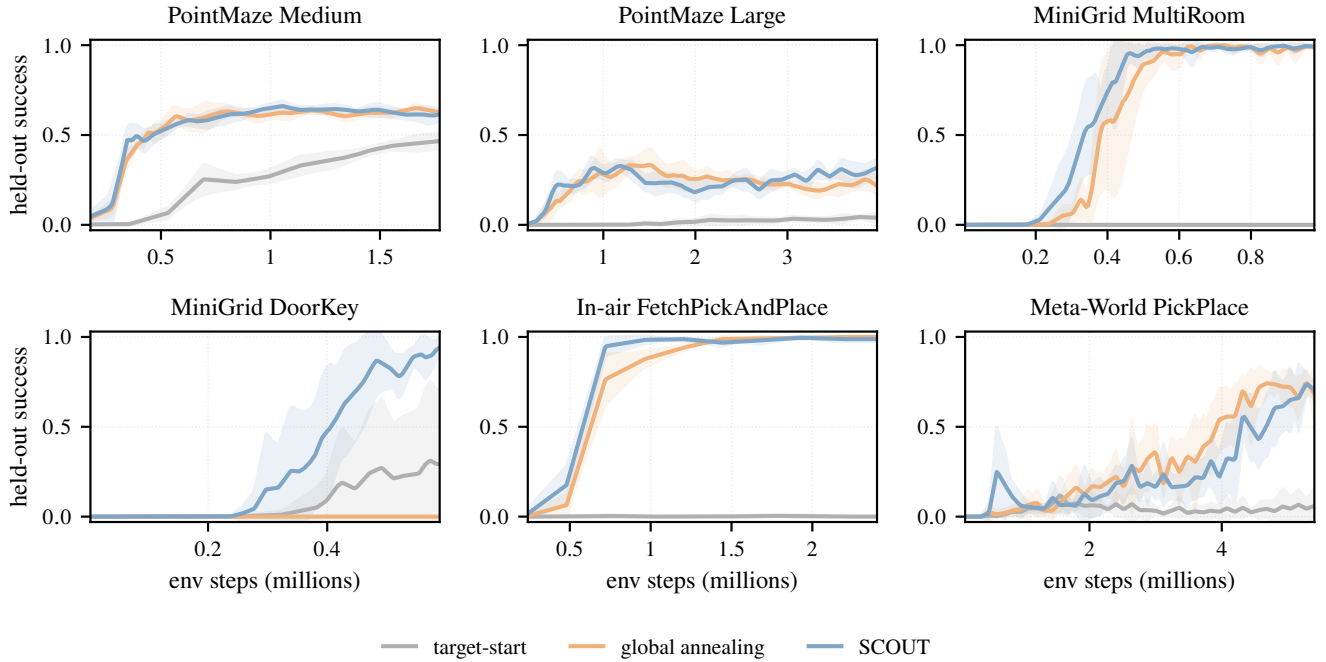


Figure 2: **Held-out target-start success across six natural settings** (mean $\pm$ s.d.; per-panel step ranges): target-start training (gray), global annealing (orange), SCOUT (blue).

global schedule fails at least one context.

A synchronized schedule gives both contexts the same number of assisted episodes. One context in this construction needs a single assisted episode and the other needs K (full proof in the supplement). The result rules out synchronized global schedules, but not population-level curricula that keep several levels available at once. We therefore include an asynchronous global mixture in the experiments. This construction is an expressivity example, not an optimality proof for SCOUT.

Experimental Setup

We evaluate SCOUT on six natural settings and three pacing conflict settings. The conflict settings include one constructed conflict, a pair of unmodified Meta-World tasks, and a constructed *within-group* conflict in which pacing heterogeneity occurs within a single labeled group (Results). We match the policy learner to each environment family but keep the controller unchanged. We use off-policy SAC+HER for goal-conditioned continuous navigation and Fetch manipulation, on-policy PPO with a partial-observation convolutional encoder for MiniGrid (Chevalier-Boisvert et al. 2023), and SAC+HER for Meta-World PickPlace (Yu et al. 2020). Thus, the settings test the same success-only allocation rule with both off-policy and on-policy optimizers. We build scaffold operators for each family: geodesic or route-based reset progress for navigation, stage-based levels for DoorKey, and reverse-curriculum starts for manipulation. Each bank contains 100 training and 50 held-out contexts. The MiniGrid settings and Meta-World conflict instead use 200 training and 100 held-out

contexts. Observations are low-dimensional goal-conditioned or state vectors, except for MiniGrid’s partial-view CNN input.

The constructed pacing conflict combines easy on-table PickAndPlace contexts with hard, long-range in-air contexts (Plappert et al. 2018). We remove the middle-difficulty range, so the two groups require assistance to be removed at different rates. All methods within each setting share the policy learner, context bank, rollout workers, reward, and training budget. We selected SCOUT’s controller hyperparameters on development seeds and fixed them across all settings. Evaluation always uses held-out contexts at $\ell = 0$. The main comparisons use five or six seeds per setting. The primary metric is area under the held-out target-start learning curve (AUC). AUC values are comparable only within the same task. We also report final success and the number of seeds solved. We report worst-group values for conflicts. Sign tests are descriptive; paired seed-bootstrap intervals provide the main uncertainty estimates.

Baselines. All conditions differ only in scaffold-level selection: target-start training, random scaffold sampling, global annealing, and a synchronized reverse curriculum. We also include the strongest fixed level ($\ell = 2$, “fixed mid”) and an asynchronous global mixture for the conflict settings. The conflict settings include five more conditions that test allocation directly. The *group frontier* runs SCOUT’s fixed rule separately for each oracle group. This baseline has more evidence, not less granularity: it uses labels that SCOUT never sees and clears the evidence gate $\sim |\text{group}| \times$ faster. PLR- and ALP-style teachers sample all (x, ℓ) cells without a frontier.

Table 1: **Scaffold access across six natural settings.** Median held-out AUC, paired $\Delta S-T$ (median of per-seed SCOUT—target differences), final success (Fin.), and seeds solved (≥ 0.5 ; Solv. S/T; n/a: neither). Global and Random are access baselines without per-context allocation. DoorKey’s AUC window closes before slow seeds solve, so its primary metrics are Fin. and Solv.

Task	Learner	Target AUC	Global AUC	Random AUC	SCOUT AUC	$\Delta S-T$ AUC	Fin. T	Fin. S	Solv. S/T
PointMaze Medium	SAC+HER	0.254	0.559	0.407	0.559	+0.314	0.62	0.63	5/5
PointMaze Large	SAC+HER	0.019	0.230	0.137	0.245	+0.226	0.19	0.30	n/a
MiniGrid MultiRoom	PPO	0.000	0.614	0.440	0.633	+0.633	0.00	1.00	6/0
MiniGrid DoorKey	PPO	0.000	0.000	0.363	0.300	+0.268	0.46	0.95	6/3
In-air FetchPickAndPlace	SAC+HER	0.002	0.791	0.789	0.840	+0.838	0.00	1.00	5/0
Meta-World PickPlace	SAC+HER	0.028	0.298	0.167	0.240	+0.172	0.02	0.73	6/0

The RFCL- and ACED-style advance-only rules reproduce the corresponding prior mechanisms without retreat.

Trajectory-derived scaffold experiments. We replace the hand-built operator for PointMaze Large, MiniGrid DoorKey, and in-air FetchPickAndPlace with the trajectory-derived construction from the Method section to show that the scaffold axis need not be hand-designed. The reference trajectories come from a shortest-path planner, a breadth-first solver, and a scripted pick-and-place controller, one for each family. *Trajectory-derived global annealing* and *trajectory-derived random scaffolding* apply global and random schedules to the same ladders. This comparison separates ladder quality from local pacing.

Results

Scaffold access improves sparse learning across six settings. SCOUT achieves higher held-out target-start AUC than the target-start baseline across all six natural settings, with every paired-seed difference positive (Table 1; Fig. 2). These gains cover two regimes. Scaffold access enables exploration in the first regime where target-start training does not learn: in-air FetchPickAndPlace, MiniGrid MultiRoom, and Meta-World PickPlace. Scaffolds improve sample efficiency in the second regime where target-start training learns slowly or unreliably: the two PointMazes and DoorKey.

Random scaffold sampling and global annealing also benefit from easier starts. Access is the main benefit when contexts learn at similar rates, and no scheduler performs best in every setting (Table 1). SCOUT’s distinguishing benefit, allocation, emerges under the pacing conflicts studied next.

Per-context allocation improves performance under pacing conflict. The constructed PickAndPlace conflict realizes the tradeoff of Proposition 1. No tested synchronized schedule solves both of its groups in our experiments. SCOUT reaches near-perfect final success on both groups with separate frontiers. Global annealing and the synchronized reverse curriculum solve only the on-table group (Fig. 3).

SCOUT improves median worst-group AUC by nearly $3\times$, from 0.123 for the strongest fixed scaffold to 0.354. The paired difference is positive in all six runs (sign test, $p = 0.031$). A second six-seed block (the *replication block*) gives the same result, with a paired difference of $+0.24$ [0.18, 0.38].¹

¹Bracketed intervals, here and throughout, are paired seed-

The asynchronous global mixture reaches 0.022, below the strongest fixed scaffold. Thus, multilevel sampling without allocation does not help.

The conflict arises on unmodified tasks. We combine the standard Meta-World push and shelf-place task definitions, without modification, in one bank. We use our scaffold operator, SAC+HER learner, and fixed controller. Push is an on-surface task that tolerates assistance. Shelf-place does not learn from the target start but can learn from grasped starts. Shelf-place success under global annealing falls to 0.000 after a temporary rise (≤ 0.44) with a five-level scaffold. SCOUT maintains shelf-place success at 0.52–0.68 and wins the paired worst-group comparison on all six seeds (sign test $p = 0.031$; Fig. 3, panel 4). Deterministic selectors fail on shelf-place while push remains high. The coverage baselines, random and the mixture, tie SCOUT on worst-group AUC but have much more variable final performance (mixture 0.38 ± 0.24 , random 0.48 ± 0.19 , and SCOUT 0.61 ± 0.06). The global schedules solve both tasks with a nine-level ladder. Per-context allocation therefore makes performance less sensitive to a coarse scaffold axis.

Does allocation need to be per context? Both conflicts align with known groups, so group-level pacing might be sufficient. We test this with the oracle-label *group frontier* (Baselines). SCOUT is more sample-efficient on the hard group in the constructed conflict (median worst-group AUC 0.378 vs 0.229 on the replication block; paired $\Delta +0.18$ [0.04, 0.29]), though both reach the endpoint (0.97 vs 0.95 worst-group final). The group frontier wins on six of six seeds in the Meta-World conflict ($\Delta +0.10$ [0.03, 0.22] worst-group AUC) and has the best worst-group final (0.72; SCOUT: 0.61). Pooling works when labels track the learning differences. PLR- and ALP-style teachers over all (x, ℓ) cells fail or are unstable on the hard group in both conflicts, so cell-level prioritization alone does not resolve cross-group competition.

Pooled pacing fails under within-group heterogeneity. All contexts in this PickAndPlace bank have the same nominal label (in-air), but goal height is bimodal. The band is hidden from every method. The low band dominates the pooled statistics, so a pooled frontier advances before the high-band contexts are ready. It ends with 0.00 high-band success on

bootstrap 95% confidence intervals of the median paired difference (10,000 seed resamples).

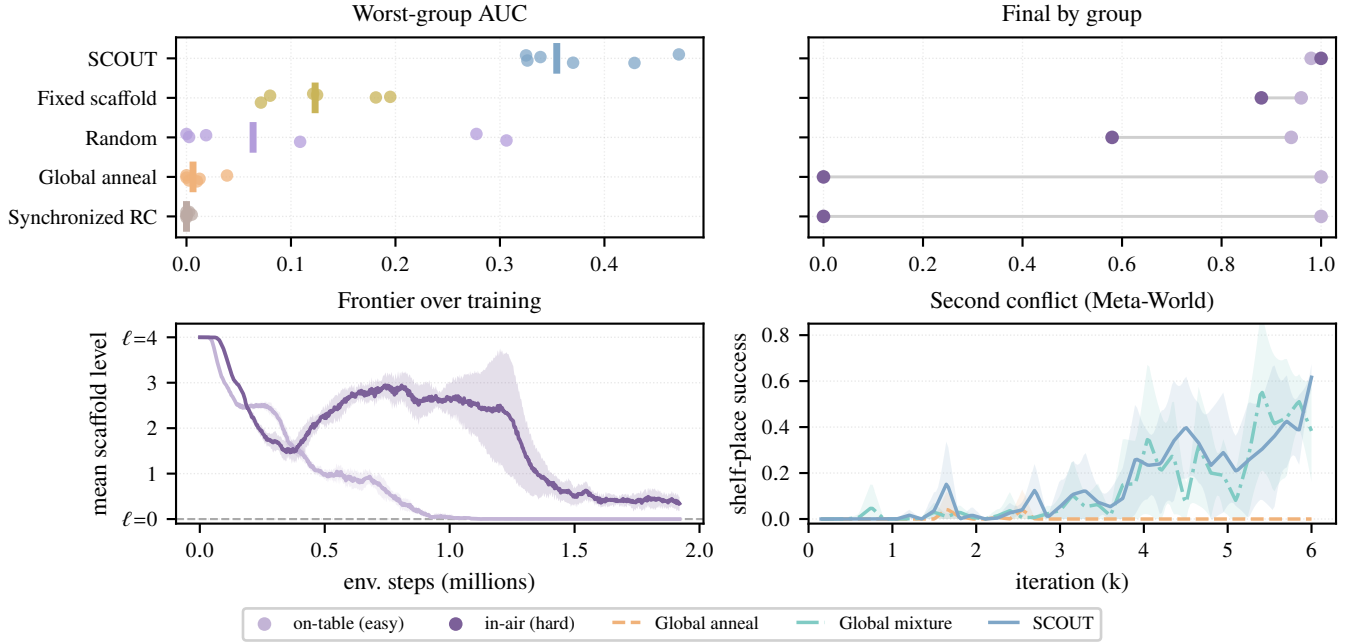


Figure 3: **Local frontiers under pacing conflicts.** Panels 1–3: constructed PickAndPlace conflict. **1:** worst-group AUC (dots seeds, ticks medians). **2:** final success by group; global schedules fail on the hard group, while SCOUT solves both. **3:** SCOUT’s mean scaffold level by group (mean $\pm$ s.d.): easy contexts advance quickly, hard ones retain assistance longer. **4:** the Meta-World conflict, shelf-place held-out success (mean $\pm$ s.d.); global annealing falls to zero, while SCOUT and the mixture improve.

Table 2: **Within-group conflict** (six seeds; goal height bi-modal inside one label): median worst-band AUC, paired Δ (SCOUT – condition). SCOUT wins the paired worst-band AUC comparison on six of six seeds against every condition.

Condition	Worst-band AUC	Worst-band final	Δ [CI]
SCOUT	0.701	0.91 ± 0.10	(ref.)
Pooled frontier (oracle)	0.000	0.22 ± 0.35	+0.69 [0.47, 0.74]
RFCL advance rule	0.396	0.93 ± 0.08	+0.30 [0.17, 0.48]
ACED-style advance	0.553	0.95 ± 0.07	+0.16 [0.11, 0.27]
PLR cells	0.615	0.92 ± 0.14	+0.08 [0.05, 0.18]
ALP cells	0.551	0.85 ± 0.20	+0.14 [0.06, 0.25]
Global annealing	0.527	0.96 ± 0.06	+0.20 [0.09, 0.27]
Asynchronous global mixture	0.451	0.83 ± 0.18	+0.29 [0.17, 0.39]

four of six seeds (median worst-band AUC 0.000). SCOUT succeeds on both bands and has higher worst-band AUC than every tested alternative on all six seeds (Table 2; Fig. 4, right). Every other label-free rule eventually succeeds on the high band (0.83–0.96 final), but SCOUT is more sample-efficient and reliable. Group labels are insufficient when heterogeneity occurs within a group.

The local frontier explains the conflict result. An RFCL-style ablation (Tao et al. 2024) that keeps the per-context trajectory frontier but samples contexts uniformly and disables antistall matches SCOUT on the homogeneous settings and the constructed conflict. The two are statistically indistinguishable over eight paired seeds (worst-group AUC 0.75 vs 0.77). We therefore attribute the advantage on the conflict to the local frontier, rather than to SCOUT’s calibrated sampling or antistall. The bare frontier again ties SCOUT on worst-group

AUC in the Meta-World conflict. However, its worst-group final has the largest variance of any condition (0.37 ± 0.28 vs 0.61 ± 0.06). Antistall and calibrated sampling therefore have little effect on the mean on coarser ladders but reduce the risk of failure.

Component-matched external rules support the same conclusion. The advance-only RFCL rule ties SCOUT on the easier constructed ladder (0.98 vs 0.97 worst-group final), but reaches only 0.12 on the coarse Meta-World ladder (SCOUT: 0.61). ACED-style advance performs worse on both conflicts (zero of six paired seeds each). Both rules solve the within-group high band but learn more slowly than SCOUT (Table 2). Bidirectional pacing matters most on coarse ladders; the supplement reports full component comparisons.

Comparison with official RFCL. Official RFCL (Tao et al. 2024) learns faster than SCOUT under an aligned PointMaze Large protocol (AUC 0.34 vs 0.20) but reaches a similar endpoint (0.31 vs 0.34); target-start SAC+HER remains at 0.00. RFCL uses demonstrations, an ensemble critic, and a higher update-to-data ratio; SCOUT uses none. The supplement gives the full protocol and curves.

Trajectory-derived scaffolds reduce manual level design. We replace the hand-built scaffold axis for PointMaze Large, MiniGrid DoorKey, and in-air FetchPickAndPlace with the trajectory-derived construction (Method) and train with the unchanged controller. The trajectory-derived scaffold retains or exceeds the hand-built benefit in all three settings (R_{retain} of 1.21 [1.05, 1.44], 1.47 (DoorKey unstable; point estimate), and 1.06 [1.05, 1.06]), improves over target-start training on

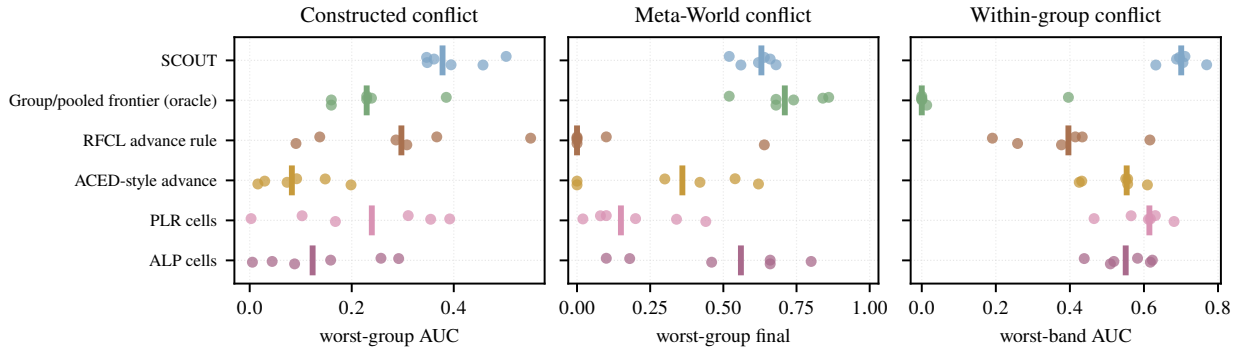


Figure 4: **Pacing granularity must match where heterogeneity lives** (per-seed dots, median ticks). The oracle frontier pools by group (left, middle) or across the single group (right). SCOUT leads the constructed conflict; the group frontier leads Meta-World; only the pooled frontier fails on the within-group hard band.

every seed, and has a 0% invalid-reset rate. Replacing the hand-built scaffold also preserves the constructed conflict result under the same controller, learner, and budget. SCOUT outperforms every baseline in all six paired seeds (worst-group AUC 0.73 vs 0.55 for trajectory-derived annealing). The global baselines eventually solve both groups on this ladder, so the remaining difference is sample efficiency. The construction still requires one successful reference trajectory per training context and simulator state restoration. Ladders re-derived from the trained policy’s own successes (policy-derived scaffolds with fallback support) also retain the hand-built benefit; details and exact fallback rates appear in the supplement.

The need for allocation depends on the base learner. A contrastive critic (Eysenbach et al. 2022) removes the PointMaze Large access bottleneck (0.48 target-start vs 0.41 with SCOUT), but not the in-air Fetch bottleneck (0.20 vs 0.24; SCOUT+SAC+HER: 1.00). Reset curricula help when access, rather than the base learner, limits learning.

Discussion

SCOUT uses binary rollout success, which is available with any learner and produces frontiers that are easy to inspect.

When SCOUT applies. SCOUT needs informative bottleneck states, an ordered assistance axis, and the ability to restore selected states. Per-context pacing matters when contexts learn at different rates. A global schedule remains competitive when they learn at similar rates. Reliable task-aligned labels favor one pooled frontier per group; SCOUT is appropriate when labels are absent or heterogeneity may occur within groups. The method does not apply without reset access, as on physical robots without instrumented resets.

Limitations and future work. The frequency of two-sided conflicts is unknown. Longer horizons may require more levels, while large banks provide less evidence per context and may require grouping. Per-context pacing may also help with embodiment-level heterogeneity (Yang, Finn, and Sadigh 2026).

Conclusion

Reset curricula for sparse-reward learning make two decisions: which intermediate states to train from, and how quickly to remove their assistance. A global pace is competitive when contexts learn at similar rates, but it can fail when progress differs, and aggregate AUC can conceal the failure. SCOUT adapts each context’s pace from binary rollout success alone. It advances after sustained success, restores assistance after failure, and probes a harder start when progress stalls. It requires no group labels and remains strong whether heterogeneity follows task groups or appears within them. Curriculum pace should match the scale at which learning progress differs.

References

- Andrychowicz, M.; Wolski, F.; Ray, A.; Schneider, J.; Fong, R.; Welinder, P.; McGrew, B.; Tobin, J.; Abbeel, P.; and Zaremba, W. 2017. Hindsight Experience Replay. In *Advances in Neural Information Processing Systems (NeurIPS)*, 5048–5058.
- Bengio, Y.; Louradour, J.; Collobert, R.; and Weston, J. 2009. Curriculum Learning. In *International Conference on Machine Learning (ICML)*, 41–48.
- Chevalier-Boisvert, M.; Dai, B.; Towers, M.; de Lazcano, R.; Willems, L.; Lahlou, S.; Pal, S.; Castro, P. S.; and Terry, J. 2023. Minigrid & Miniworld: Modular & Customizable Reinforcement Learning Environments for Goal-Oriented Tasks. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.
- Dai, S.; Hofmann, A.; and Williams, B. C. 2021. Automatic Curricula via Expert Demonstrations. In *Deep Reinforcement Learning Workshop, NeurIPS*.
- Dennis, M.; Jaques, N.; Vinitzky, E.; Bayen, A.; Russell, S. J.; Critch, A.; and Levine, S. 2020. Emergent Complexity and Zero-shot Transfer via Unsupervised Environment Design. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Diaz-Bone, L.; Bagatella, M.; Hübotter, J.; and Krause, A. 2025. DISCOVER: Automated Curricula for Sparse-Reward Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Duan, Y.; Wang, Y.; Qiu, W.; and Zhu, H. 2025. Learning from Demonstrations via Capability-Aware Goal Sampling. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Eysenbach, B.; Zhang, T.; Levine, S.; and Salakhutdinov, R. 2022. Contrastive Learning as Goal-Conditioned Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Florensa, C.; Held, D.; Geng, X.; and Abbeel, P. 2018. Automatic Goal Generation for Reinforcement Learning Agents. In *International Conference on Machine Learning (ICML)*, 1514–1523.
- Florensa, C.; Held, D.; Wulfmeier, M.; Zhang, M.; and Abbeel, P. 2017. Reverse Curriculum Generation for Reinforcement Learning. In *Conference on Robot Learning (CoRL)*, 482–495.
- Graves, A.; Bellemare, M. G.; Menick, J.; Munos, R.; and Kavukcuoglu, K. 2017. Automated Curriculum Learning for Neural Networks. In *International Conference on Machine Learning (ICML)*, 1311–1320.
- Haarnoja, T.; Zhou, A.; Abbeel, P.; and Levine, S. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *International Conference on Machine Learning (ICML)*, 1856–1865.
- Jiang, M.; Grefenstette, E.; and Rocktäschel, T. 2021. Prioritized Level Replay. In *International Conference on Machine Learning (ICML)*, 4940–4950.
- Klink, P.; D’Eramo, C.; Peters, J.; and Pajarinen, J. 2020. Self-Paced Deep Reinforcement Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 9216–9227.
- Mattiisen, T.; Oliver, A.; Cohen, T.; and Schulman, J. 2020. Teacher–Student Curriculum Learning. *IEEE Transactions on Neural Networks and Learning Systems*, 31(9): 3732–3740.
- Narvekar, S.; Peng, B.; Leonetti, M.; Sinapov, J.; Taylor, M. E.; and Stone, P. 2020. Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey. *Journal of Machine Learning Research (JMLR)*, 21(181): 1–50.
- Parker-Holder, J.; Jiang, M.; Dennis, M.; Samvelyan, M.; Forster, J.; Grefenstette, E.; and Rocktäschel, T. 2022. Evolving Curricula with Regret-Based Environment Design. In *International Conference on Machine Learning (ICML)*, 17473–17498.
- Plappert, M.; Andrychowicz, M.; Ray, A.; McGrew, B.; Baker, B.; Powell, G.; Schneider, J.; Tobin, J.; Chociej, M.; Welinder, P.; Kumar, V.; and Zaremba, W. 2018. Multi-Goal Reinforcement Learning: Challenging Robotics Environments and Request for Research. *arXiv preprint arXiv:1802.09464*.
- Portelas, R.; Colas, C.; Hofmann, K.; and Oudeyer, P.-Y. 2020. Teacher Algorithms for Curriculum Learning of Deep RL in Continuously Parameterized Environments. In *Conference on Robot Learning (CoRL)*, 835–853.
- Tao, S.; Shukla, A.; Chan, T.-k.; and Su, H. 2024. Reverse Forward Curriculum Learning for Extreme Sample and Demonstration Efficiency in Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.
- Wang, X.; Ren, G.; Dai, T.; Hu, B.; Huang, S.; Zhang, W.; and Liu, H. 2026. ACDC: Adaptive Curriculum Planning with Dynamic Contrastive Control for Goal-Conditioned Reinforcement Learning in Robotic Manipulation. In *International Conference on Automated Planning and Scheduling (ICAPS)*.
- Wöhlke, J.; Schmitt, F.; and van Hoof, H. 2020. A Performance-Based Start State Curriculum Framework for Reinforcement Learning. In *Proceedings of the 19th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, 1503–1511.
- Yang, J.; Finn, C.; and Sadigh, D. 2026. Data Analogies Enable Efficient Cross-Embodiment Transfer. *arXiv preprint arXiv:2603.06450*.
- Yin, P.; Westenbroek, T.; Zhang, Z.; Tran, J.; Dagnino, I.; Shilamkar, E.; Mbiziwo-Tiapo, N.; Bagaria, S.; Liu, X.; Mullins, G.; Kolobov, A.; and Gupta, A. 2026. Emergent Dexterity via Diverse Resets and Large-Scale Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.
- Yu, T.; Quillen, D.; He, Z.; Julian, R.; Hausman, K.; Finn, C.; and Levine, S. 2020. Meta-World: A Benchmark and Evaluation for Multi-Task and Meta Reinforcement Learning. In *Conference on Robot Learning (CoRL)*, 1094–1100.

SCOUT: Per-Context Reset Curricula for Sparse-Reward Reinforcement Learning

Supplementary Results

Siddharth Aphale¹, Ayushman Singh²

¹Stanford University ²Sesame AI
saphale@stanford.edu, ayushman@sesame.com

This appendix gives the experimental protocols, reproducibility details, scaffold-construction procedures, proof of Proposition 1, supporting conflict results, and controller ablations.

A Experimental details, protocols, and reproducibility

Table A1 lists the nine settings. Table A2 gives the selection rule for each baseline.

Table A1: Experimental settings. Six natural settings (top) test scaffold access; the three pacing conflict settings (bottom) test allocation and its granularity. HER applies only to the goal-conditioned SAC settings.

Setting	Scaffold operator	Learner	Observation	Train / held-out
PointMaze Medium	geodesic reset progress	SAC+HER	low-dim goal-cond.	100 / 50
PointMaze Large	geodesic reset progress	SAC+HER	low-dim goal-cond.	100 / 50
MiniGrid MultiRoom	route-based reset progress	PPO	CNN, partial	200 / 100
MiniGrid DoorKey	stage-based reset progress	PPO	CNN, partial	200 / 100
In-air FetchPickAndPlace	reverse-curriculum starts	SAC+HER	low-dim goal-cond.	100 / 50
Meta-World PickPlace	reverse-curriculum starts	SAC+HER	low-dim goal-cond.	100 / 50
PickAndPlace pacing conflict	reverse-curriculum starts	SAC+HER	low-dim goal-cond.	100 / 50
Meta-World push × shelf-place conflict	reverse-curriculum starts	SAC+HER	low-dim goal-cond.	200 / 100
PickAndPlace within-group conflict	reverse-curriculum starts	SAC+HER	low-dim goal-cond.	100 / 50

Scaffold operators. We define a hand-built $G(x, \ell)$ for each environment family. In PointMaze, the reset places the point partway along the geodesic path from the context’s start to its goal. Larger ℓ starts closer to the goal. MiniGrid MultiRoom resets the agent along the route to the goal. DoorKey maps levels to task stages: carrying the key, at the locked door with the key, past the opened door, and next to the goal. Level $\ell=0$ is the original task. Manipulation uses reverse-curriculum starts. The object is placed partway along a solved trajectory toward the goal. For in-air goals, the reset also creates a grasped configuration. The levels move back toward the target start, which is $\ell=0$.

Controller coefficients. Table A3 lists the frozen controller constants, which we selected on conflict development seeds (Appendix D). Scaffold axes use five levels with fractions $\{0, 0.25, 0.5, 0.75, 0.9\}$ on the continuous settings and $\{0, 0.25, 0.5, 0.75, 1.0\}$ on MiniGrid. All settings use sparse 0/−1 rewards, with HER’s future strategy on the goal-conditioned settings (Table A1). Algorithm 1 of the main paper gives the full controller loop.

Controller-window semantics. One *controller window* (equivalently controller step) is one training iteration. The controller samples active cells (Eq. 1 of the main paper), collects rollouts, updates per-cell statistics, and runs one frontier-transition pass (Eq. 2 of the main paper). It checks every context for eligibility in each window. A transition direction must hold for two consecutive eligible windows before the frontier moves. Per-cell statistics persist when a frontier leaves a cell and later returns. A cell that has never been visited starts with no evidence, so it cannot transition until it has accumulated 15 rollouts. To form the sampling distribution, the controller applies the softmax, the exploration floor, and then the per-cell cap. It redistributes excess probability proportionally and renormalizes.

Table A2: Baseline scaffold-level selection rules. Conditions share their setting’s full training stack, differing only in reset-cell choice.

Baseline	Selection mechanism	Constants
Target-start training	every rollout at $\ell = 0$ (no scaffold)	—
Fixed scaffold level [†]	every rollout at one constant level	$\ell = 2$ (“fixed mid”)
Random scaffold sampling	$\ell \sim \text{Unif}\{0, \dots, L\}$ per rollout, competence-independent	—
Global annealing	shared schedule $\ell(t) = \text{round}(L \max(0, 1 - t/T_{\text{anneal}}))$	T_{anneal} : 2000 constructed / 3000 Meta-World
Synchronized reverse curriculum	every context’s start progress decays $0.9 \rightarrow 0$ linearly, identically	—
Asynchronous global mixture [†]	time-varying categorical over levels; $\ell=0$ mass grows with aggregate-success EMA	rate 0.05; mass $0.1 \rightarrow 0.7$; easy floor 0.2
RFCL-style local frontier	SCOUT’s frontier transition, uniform context sampling, antistall disabled	frozen SCOUT thresholds
Group frontier ^{†‡}	SCOUT’s frozen rule, one pooled frontier and statistics per oracle group	frozen SCOUT hyperparameters
PLR cells [†]	rank-prioritized teacher over all (x, ℓ) cells by per-cell mean $ \text{TD} $, staleness-mixed	$\beta=0.5, \rho=0.1, \alpha=0.3$, SCOUT floor/cap
ALP cells [†]	learning-progress teacher over all cells, score $ s - s_{\text{slow}} $	SCOUT temperature, floor, cap
RFCL advance rule [†]	advance iff ≥ 3 of last 5 rollouts succeed; no retreat or antistall	struggling-context weight $w = (1 - \delta) + 0.1$
ACED-style advance [†]	threshold-triggered advance only; no retreat or antistall	SCOUT τ_{high}
Anneal + balanced replay ^{†‡}	global annealing; SAC minibatch draws fixed per-group quotas	minibatch/ G per group; empty group cedes its quota

[†]: conflict settings only. [‡]: consumes oracle group labels. Balanced replay differs in the minibatch, not the reset choice.

Seeds and banks. We use five seeds for the settings that reuse earlier development banks: the two PointMazes and in-air FetchPickAndPlace. We use six seeds for MiniGrid MultiRoom, DoorKey, Meta-World PickPlace, and the PickAndPlace conflict. A second six-seed block confirms the PickAndPlace conflict. The primary Meta-World conflict protocol also uses six seeds, with smaller blocks for the robustness checks (Appendix C.3, Table A11). The trajectory-derived and policy-derived experiments (Table A7; Table A8) use three seeds for the continuous settings and six for DoorKey. Other experiments state their seed counts with their results. The code release will include the bank-construction seeds and SHA-256 hashes.

Metrics. The primary metric is area under the held-out target-start learning curve within each task. AUC is comparable only within a task because episode length varies across levels. We also report final held-out success and the number of seeds solved. Held-out success is *success-once*: an episode succeeds if the environment’s success indicator is true at any step, rather than only at termination. For the within-task AUC experiments (Table 1 of the main paper; Table A7), we report medians to reduce the influence of an occasional non-solving seed. Conflict tables report means of the per-seed worst-group minima (Table A10), except that the main constructed-conflict results use medians. Learning-curve figures show mean $\pm$ standard deviation.

Evaluation counts and inference convention. Each evaluation point averages a fixed episode budget over the held-out contexts at $\ell = 0$: 100 episodes on the constructed and within-group conflicts (50 contexts $\times$ 2 each), 200 on the Meta-World conflict (100×2), 100 on the PointMazes, and 50 on the MiniGrid settings. For paired conflict comparisons, the primary uncertainty estimate is a seed-bootstrap 95% confidence interval for the median paired difference in worst-group AUC. We use 10,000 resamples of the six paired seeds. We report exact two-sided sign tests descriptively as wins out of six. With six paired seeds, the smallest attainable two-sided p -value is 0.031. The primary family contains 27 such tests: SCOUT against every baseline on the two conflicts and the within-group experiment. No individual sign test survives a Holm–Bonferroni correction for a family of this size. We therefore base the main comparisons on the bootstrap intervals. Screens and ablations are exploratory and reported per comparison.

Compute. All training runs are CPU-only by protocol (device="cpu" in every training configuration). GPU execution changes the numerical environment and is not part of the protocol. We ran experiments in three Linux environments: a 20-core NVIDIA DGX Spark (Grace aarch64, 121 GiB unified memory), a single cloud H100 node whose GPU remains idle under this protocol (221 GiB host RAM), and CPU nodes on a university SLURM cluster (20 cores and 32 GB per job). One 2400-iteration Fetch conflict run takes about 40 minutes on the 20-core machine and up to five hours on the cluster CPU nodes. Six to seven concurrent 16-worker runs use all 20 cores. The software versions are Python 3.11–3.12, PyTorch 2.12.1, Gymnasium 1.3.0, Gymnasium-Robotics 1.4.2, MuJoCo 3.10.0, Meta-World 3.0.0, MiniGrid 3.1.0, and NumPy 2.4–2.5.

Table A3: Frozen controller constants (shared across every setting and condition).

Component	Symbol	Value
Sampling score weights (Eq. 1, main paper)	$\lambda_h / \lambda_p / \lambda_{\text{stale}}$	1.0 / 0.2 / 0.1
Softmax temperature	τ	0.5
Exploration floor / per-cell cap	$\epsilon / q_{\text{max}}$	0.1 / 0.5
Transition thresholds (Eq. 2, main paper)	$\tau_{\text{low}} / \tau_{\text{high}}$	0.15 / 0.8
Evidence gate	n_{min}	15 rollouts
Transition cooldown	—	3 controller windows
Antistall limit	N_{stale}	4
Success EMA	β	0.2

Learner hyperparameters. The policy learners are custom single-file implementations adapted from CleanRL. We fixed their hyperparameters before running the test seeds and use the same values for every condition within a setting (Table A4). Training length, replay capacity, exploration constants, and worker count vary by setting and episode horizon. The code release will include these values in the run configurations.

Table A4: Learner hyperparameters (fixed a priori; shared by every condition within a setting).

<i>SAC / SAC+HER (PointMaze, Fetch, Meta-World)</i>	
Networks	MLP (256, 256), ReLU; double-Q critic; tanh-Gaussian actor
Optimizer / learning rate	Adam, 3×10^{-4} (actor, critics, entropy)
Discount / target-update rate	0.99 / 0.005
Entropy temperature	auto-tuned; target $-\dim(\mathcal{A})$, initial 0.2
Minibatch / warmup	256 / 1,000 transitions
Gradient steps per iteration	16 (32 on in-air FetchPickAndPlace)
Replay capacity	2×10^5 – 6×10^5 (per setting)
HER	future strategy, $k = 4$ (goal-conditioned settings)
Exploration	random-action prob. 0.3, action noise 0.2 (0.1 / 0.1 on the PointMazes)
<i>PPO (MiniGrid)</i>	
Encoder	Conv 3→16 and 16→32 (2×2), MLP (128, 128), Tanh
Optimizer / learning rate	Adam, 2.5×10^{-4}
Discount / GAE λ	0.99 / 0.95
Clip / epochs / minibatch	0.2 / 4 / 1,024
Entropy / value coef. / grad. norm	0.01 / 0.5 / 0.5
Episodes per iteration	16
<i>Contrastive RL (strong-learner comparison)</i>	
Critic	InfoNCE over in-batch negatives, representation dim 64
Regularizer	logsumexp penalty 0.1
Positives / actor	HER-future achieved goals / SAC actor and entropy

A.1 Baseline protocols, tuning provenance, and external-rule component map

Table A2 gives the selection mechanism and constants for every baseline. Contexts are drawn uniformly unless stated otherwise.

Official RFCL protocol. The official implementation (Tao et al. 2024) runs on the PointMaze Large bank with one hundred state-only demonstrations, sparse reward, and the shared held-out evaluation. Each of the three seeds runs for one million steps. Fig. A1 shows the held-out learning curves used in the main-paper comparison.

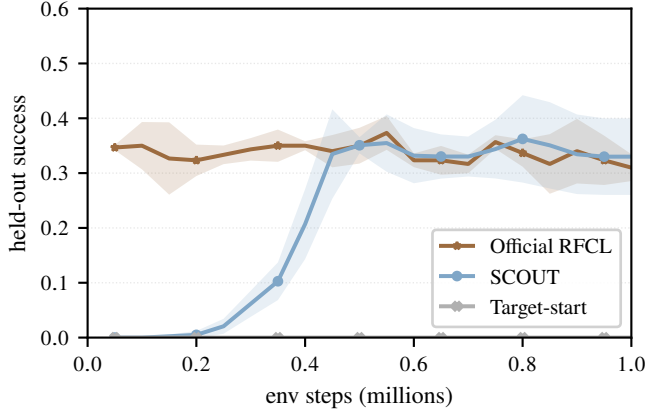


Figure A1: Aligned official RFCL comparison on PointMaze Large (held-out success-once, mean $\pm$ s.d., three seeds, shared 1M-step budget). Official RFCL is near its plateau by the first checkpoint (50k steps), consistent with its ensemble critic and high update-to-data ratio; SCOUT reaches the same endpoint with plain SAC+HER and no demonstrations; target-start SAC+HER stays at zero.

Tuning provenance. Controller hyperparameters (score weights, thresholds, $n_{\min}$, cooldown, hysteresis, antistall $N_{\text{stale}} = 4$, EMA $\beta = 0.2$) were selected on conflict development seeds (Appendix D) and frozen across every setting and condition. “Fixed mid” was the best level on the development seeds. A full level sweep confirms this choice (Table A5): $\ell = 2$ is the strongest constant on both conflicts, and no constant exceeds 0.15 worst-group final success on Meta-World. We also screened the annealing length on development seeds (Table A13). In a five-length screen on the constructed conflict, no length solves the hard group; worst-group AUC is at most 0.017 for every length. Thus, the default does not benefit from favorable selection. We set the mixture coefficients before running the test seeds. The campaign conditions add no tuned values. The group frontier inherits all fixed SCOUT hyperparameters, and the cell teachers use the repository’s existing PLR sampler defaults. We recorded every configuration before its runs began. Evaluation seeds are 42–47. Development screens use 40–42, 42–44, or 50–52. We did not select any configuration using final-test outcomes. The within-group runs of the four external rules (Table 2 of the main paper) reuse these configurations on the unchanged within-group bank. They add no tuned values and use evaluation seeds 42–47.

Table A5: Fixed-level sweep (both conflicts; worst-group AUC median / worst-group final median; levels 1–2 at six seeds where the block pre-exists, else three screen seeds).

Conflict	$\ell = 1$	$\ell = 2$ (“fixed mid”)	$\ell = 3$	$\ell = 4$
Constructed (wAUC / worst final)	0.013 / 0.40	0.131 / 0.81	0.010 / 0.00	0.000 / 0.00
Meta-World (wAUC / worst final)	0.021 / 0.15	0.024 / 0.05	0.006 / 0.00	0.003 / 0.00

B Scaffold construction: trajectory-derived and policy-derived

The trajectory-derived extractor (main paper, Method) turns one successful reference trajectory for each training context into a scaffold axis. It uses the same fixed progress quantiles across tasks, except for the terminal DoorKey quantile, and a separate state codec for each environment. We compare automatic and hand-built scaffolds using the retained gain ratio

$$R_{\text{retain}} = \frac{\text{AUC}_{\text{auto}} - \text{AUC}_{\text{true}}}{\text{AUC}_{\text{manual}} - \text{AUC}_{\text{true}}}, \quad (\text{A1})$$

Table A6: Component map: SCOUT vs. the demonstration-anchored prior rules as implemented in our component-matched baselines (main paper, Results). SCOUT’s delta is the bidirectional transition (retreat + antistall) and the general (non-demonstration) frontier index; the sampling scores are closely related.

Component	SCOUT	RFCL (Tao et al. 2024)	ACED (Dai, Hofmann, and Williams 2021)
Frontier index	task context x	demonstration	demonstration section
Assistance axis	ordered reset ladder (any semantics)	demo timestep	demo sections
Advance	success EMA $\geq \tau_{\text{high}}$, evidence + cooldown	≥ 3 of last 5 rollouts succeed	success threshold
Retreat	success EMA $\leq \tau_{\text{low}}$	none	none
Antistall	forced advance after N_{stale} idle windows	none	none
Sampling	success-calibrated softmax, floor, cap	struggling-demo weighting	uniform over active sections
Success statistic	EMA ($\beta = 0.2$), $n_{\text{min}} = 15$	rolling window (5)	running rate

This ratio is the fraction of the hand-built scaffold’s gain over target-start training that the trajectory-derived scaffold recovers. A value at or above one means that the automatic scaffold matches or exceeds the hand-built scaffold (Table A7).

Table A7: Trajectory-derived scaffolds retain the hand-built benefit. Mean held-out AUC on the block’s common window; $R_{\text{retain}} \geq 1$ (Eq. A1) means the automatic scaffold matches or exceeds the hand-built one. CI: seed bootstrap of the mean-AUC ratio; DoorKey’s is unstable, so point estimate only. References differ from Table 1 of the main paper, whose DoorKey window closes before slow seeds solve.

Setting	Target start	Manual SCOUT	Automatic SCOUT	R_{retain}	CI	Invalid
PointMaze Large	0.012	0.225	0.269	1.21	[1.05, 1.44]	0%
MiniGrid DoorKey	0.259	0.451	0.541	1.47	—	0%
In-air FetchPickAndPlace	0.002	0.801	0.845	1.06	[1.05, 1.06]	0%

State codecs. PointMaze stores the point position and velocity (the goal is fixed by the context). DoorKey stores the encoded grid, agent pose, and any carried object. FetchPickAndPlace stores the full MuJoCo state, including the actuator controls that hold a grasp and the welded gripper mocap target. Without the actuator controls, a restored in-air grasp opens and the object drops.

Offline validation. Before RL begins, we restore every extracted state and run a few settling steps. We check for finite observations, in-bounds object positions, and, for Fetch, stable grasps. All three settings have a 0% invalid-reset rate.

Reference trajectory sources. PointMaze trajectories come from a shortest-path planner. DoorKey trajectories come from a breadth-first solver that steps the environment. The solver omits the final goal-reaching step, so the most assisted level is adjacent to the goal. Fetch trajectories come from a four-phase scripted pick-and-place controller. Each source solves every training context. On DoorKey, the fixed progress quantiles recover the key, door, and goal progression without event labels.

B.1 Policy-derived scaffolds with fallback support

These experiments study post-training scaffold *recycling and refinement*. The policy that supplies the reference trajectories was first trained with a scaffold. The results show that the scaffold axis can be derived again from the policy’s own competence. They do not show that scaffolds can be discovered from the unassisted sparse-reward problem. The trajectory-derived extractor does not depend on the source of the reference trajectory. We test trajectories from the policy’s *own* first target-start success in each context. After a scaffolded training run, we roll out the trained policy from each training context’s target start. We make $k=5$ stochastic attempts and retain the shortest successful trajectory. We then pass its states through the same extractor and use the same controller. Every policy-derived ladder passes offline restore validation, with a 0% invalid-reset rate. The target-start and manual baselines are the same runs used in the trajectory-derived comparison, with the window matched to Table A7. Within this block, R_{self} compares the policy-derived and manual scaffolds.

We report a *bootstrap fraction*, the share of contexts whose ladder is not a purely unassisted self solution (bootstrapped by the scaffold-trained policy or, failing that, a scripted fallback). This fraction indicates how much each ladder depends on prior scaffolding.

Table A8: Policy-derived scaffolds (with fallback support) retain the hand-built benefit. Bootstrap fraction = (bootstrapped + scripted fallback)/ n_{train} , lower more purely policy-derived; $R_{\text{self}} \geq 1$ matches the hand-built scaffold.

Setting	Bootstrap frac.	R_{self}	> target (seeds)	Invalid
MiniGrid DoorKey	0.005	1.76	5/6	0%
PointMaze Large	0.810	1.22	3/3	0%
In-air FetchPickAndPlace	0.990	1.03	3/3	0%

Table A8 shows that policy-derived scaffolds retain the hand-built benefit in all three settings. DoorKey is nearly fully policy-derived, with 0.5% fallback, and slightly exceeds the scripted trajectory-derived ladder ($R_{\text{self}} = 1.76$ vs $R_{\text{retain}} = 1.47$). For Fetch, 99% of the trajectories come from the scaffold-trained policy and none use the scripted fallback. PointMaze Large requires more support. The policy solves only about one third of the contexts, so 65% use the scripted reference. As a result, R_{self} largely measures those fallback cells.

C Pacing conflicts: supporting results and the Meta-World conflict

This section gives the toy construction and proof behind Proposition 1, supporting analyses of the constructed conflict, and results for the Meta-World conflict.

C.1 The toy construction and proof of Proposition 1

The main paper states Proposition 1 with a one-sentence sketch. Here we give the full construction and proof. The construction represents the scheduling conflict tested in the experiments, rather than the full reinforcement learning dynamics.

Consider two contexts, a and b , and two scaffold levels, $\ell = 1$ (assisted) and $\ell = 0$ (the target start). Each context receives the same budget $H = K + 1$ training episodes, $K > 1$. Let $n_1(x)$ and $n_0(x)$ denote the number of assisted and target-start episodes assigned to context x . Assume context a succeeds at evaluation iff $n_1(a) \geq 1$ and $n_0(a) \geq K$, and context b iff $n_1(b) \geq K$ and $n_0(b) \geq 1$. Since $n_0(x) = K + 1 - n_1(x)$, these reduce to

$$a \text{ succeeds iff } n_1(a) = 1, \quad b \text{ succeeds iff } n_1(b) = K.$$

Context a needs assistance removed quickly; context b needs it retained.

Proposition 1 (synchronized global pacing can be insufficient; restated). *For every $K > 1$, there exists a two-context, two-level scaffold curriculum with per-context budget $K + 1$ such that a context-local schedule solves both contexts, while every synchronized global schedule fails at least one context.*

Proof. A synchronized global schedule gives both contexts the same assisted count n_1 ; context a requires $n_1 = 1$ and context b requires $n_1 = K > 1$, so every synchronized schedule fails at least one context. A context-local schedule assigns $n_1(a) = 1$ and $n_1(b) = K$, satisfying both under the same per-context budget. $\square$

Scope. The construction rules out synchronized global schedules, not richer population-level curricula that keep several levels available at once. For this reason, the main paper includes an asynchronous global mixture. The construction compares the schedules they can represent; it is not an optimality proof for SCOUT.

C.2 Supporting results on the constructed conflict

Constructed conflict: granularity and external-rule comparison. Table A9 reports the additional conditions on the constructed conflict. We compute all values over the replication block’s common window and pair each condition with that block’s SCOUT runs. The two blocks with the same protocol have medians of 0.354 (main paper, Results) and 0.378. Their difference is 0.024. The observed group-frontier difference is 7.5 $\times$ its between-block variation.

Table A9: Constructed conflict: added conditions (six seeds; median worst-group AUC, mean $\pm$ s.d. worst-group final, paired median Δ (SCOUT – condition); replication-block window). ‡: consumes oracle group labels. All conditions are specified in Appendix A.1.

Condition	Worst AUC	Worst final	Δ [CI]	SCOUT wins
SCOUT	0.378	0.97 ± 0.04	(ref.)	—
Group frontier (oracle)‡	0.229	0.95 ± 0.02	+0.18 [0.04, 0.29]	5/6 ($p = 0.22$)
RFCL advance rule (no retreat)	0.297	0.98 ± 0.03	+0.13 [−0.09, 0.30]	4/6 ($p = 0.69$)
ACED-style advance (no retreat)	0.083	0.60 ± 0.24	+0.30 [0.23, 0.40]	6/6 ($p = 0.031$)
PLR cells	0.239	0.63 ± 0.37	+0.15 [0.05, 0.35]	6/6 ($p = 0.031$)
ALP cells	0.123	0.81 ± 0.36	+0.25 [0.13, 0.40]	6/6 ($p = 0.031$)

Within-group frontier dispersion. Fig. A2 shows the mechanism behind the granularity result (main paper, Results). It plots the per-window IQR of SCOUT’s frontiers within each oracle group. By the middle of training, the easy groups use a single shared level. The hard groups retain one to two levels of dispersion on the constructed conflict and up to four on shelf-place. At any time, more than half of the hard-group contexts differ from their group’s median level. A single pooled level cannot represent this variation.

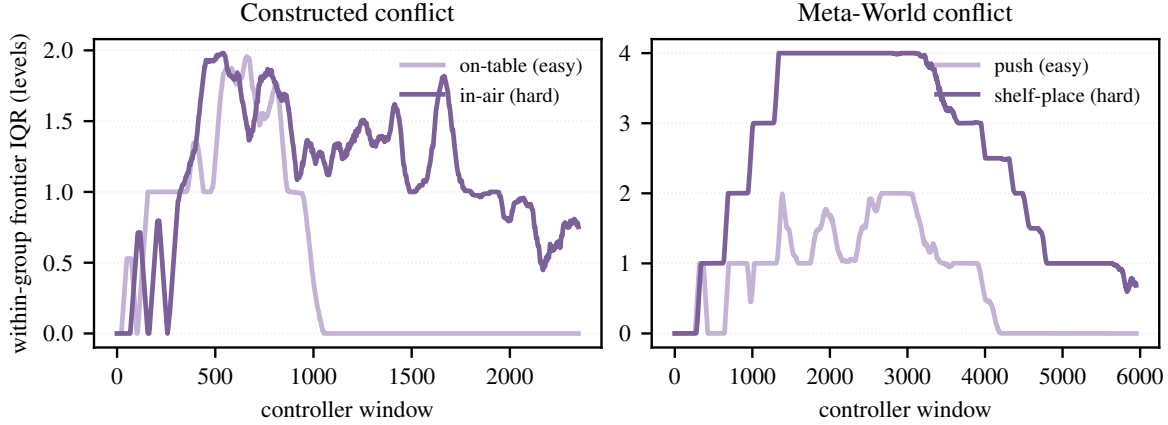


Figure A2: Within-group frontier dispersion (median over six seeds, smoothed; light = easy group, dark = hard group). SCOUT’s per-context frontiers disperse substantially inside each oracle group, most strongly in the hard groups.

What aggregate metrics hide. Fig. A3 compares aggregate and worst-group AUC. On the Meta-World conflict, aggregate held-out AUC varies by $1.2\times$ across conditions, while worst-group AUC varies by $\sim 50\times$. The three methods that fail on one group are therefore within $\sim 5\%$ aggregate AUC of the methods that maintain performance on both groups. With equal group sizes, aggregate final success preserves the ranking. Our masking claim therefore applies to aggregate AUC, not final success.

C.3 A second pacing conflict on unmodified Meta-World tasks

We screened natural context splits in MiniGrid DoorKey and Meta-World at reduced budgets, using two to six seeds. None shows the two-sided opposition of the constructed conflict. We find one consistent *one-sided* conflict and evaluate it at the full protocol below. It pairs push with shelf-place. Shelf-place does not learn from the target start but can learn from grasped starts, while push tolerates assistance.

The push $\times$ shelf-place conflict. We combine the two unmodified tasks in one bank (100 train + 50 held-out contexts per task) under the same scaffold operator, SAC+HER learner, and frozen controller as elsewhere, for 6000 iterations over six seeds (groups = tasks). Table A10 lists every condition. The main paper discusses the campaign rows: the group frontier, advance-only rules, cell teachers, and balanced-replay control. Appendix A.1 specifies their protocols. The result also replicates on a second software stack. SCOUT’s frontiers follow the same pattern as in the constructed conflict. Push reaches the target start early, while shelf-place retains assistance for most of training (Fig. A4, panel 3).

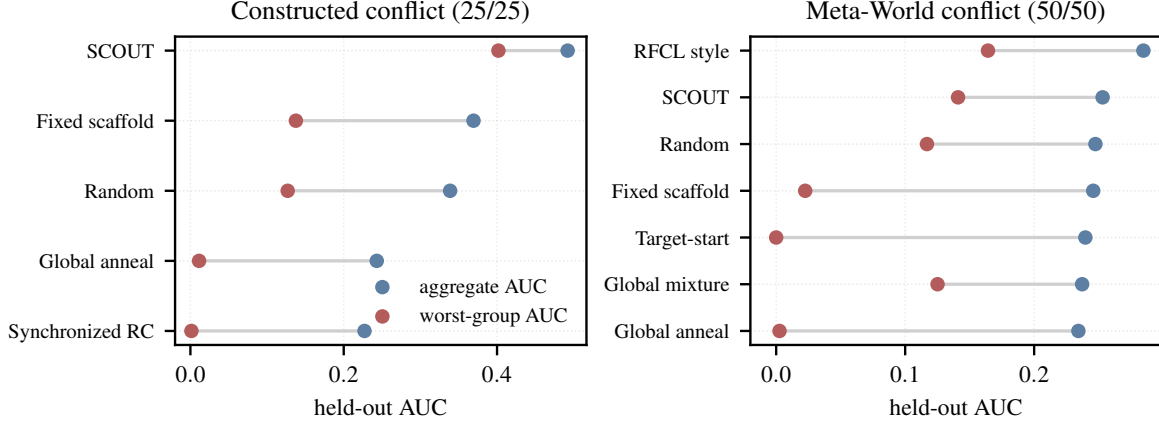


Figure A3: Aggregate AUC vs. worst-group AUC per condition (group sizes in titles). Failure on one group has little effect on aggregate AUC but sharply reduces worst-group AUC.

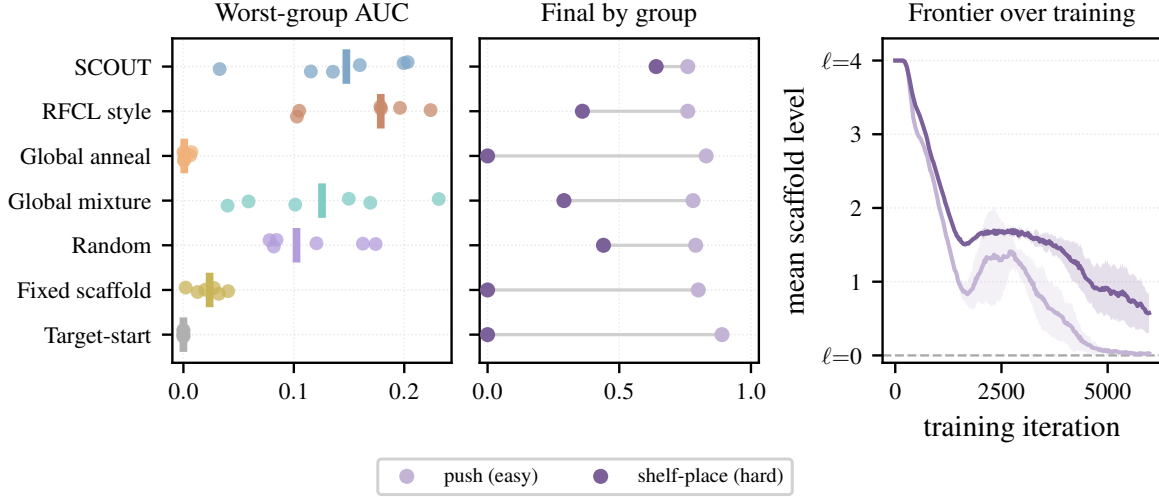


Figure A4: The Meta-World conflict, in the format of Fig. 3 of the main paper. 1: worst-group AUC. 2: final success by group; deterministic schedules fail on shelf-place. 3: mean scaffold level by group: push advances early, shelf-place retains assistance far longer.

Horizon and ladder-resolution robustness. Two follow-up blocks bound the result (Table A11). First, we increase the horizon to 9000 iterations while keeping the five-level protocol. Global annealing still ends at 0.000 shelf-place success on all three seeds. The failure therefore persists with more training. SCOUT reaches 0.680, while the mixture improves later and reaches 0.853. Second, we use a nine-level ladder at the same horizon. All methods then solve both tasks. A deterministic global schedule fails when the ladder is too coarse for one pace to serve both groups. SCOUT solves shelf-place with either ladder.

Long-horizon level count. A long-horizon DoorKey-16×16 study (automatic SCOUT, three seeds; $\approx 2.3 \times$ the 8×8 horizon) tests the same issue for scaffold access. Increasing the ladder from five to nine levels raises final success by ≈ 0.10 without reducing AUC. Thirteen levels provide little further benefit and reduce AUC because the fixed budget spends too long at each level. Longer horizons can benefit from more levels, but too many levels slow progression.

DoorKey pacing splits. We also split DoorKey-8×8 by geometry into short and long solutions. We train each group at a single fixed scaffold level. No fixed level solves both groups. The easy group remains at zero for every level and seed. The hard group reaches 0.74 at only one intermediate level. In contrast, the same controller used

Table A10: Meta-World push $\times$ shelf-place conflict (six seeds, 6000 iterations, five-level scaffold; cells are seed means; best per column bold). Worst group = within-seed minimum over the two tasks (iteration-axis AUC); “SCOUT wins” counts paired worst-group AUC wins (sign test, descriptive). ‡: consumes oracle task labels. The RFCL-style frontier keeps retreat; the RFCL advance rule is advance-only (Appendix A.1).

Condition	Worst AUC	Push final	Shelf final	Worst final	SCOUT wins
SCOUT	0.141	0.743	0.617	0.613	(ref.)
Group frontier (oracle)‡	0.256	0.840	0.757	0.720	0/6 ($p = 0.031$)
RFCL-style local frontier	0.164	0.743	0.373	0.367	3/6 ($p = 1.0$)
RFCL advance rule (no retreat)	0.042	0.763	0.167	0.123	5/6 ($p = 0.22$)
ACED-style advance (no retreat)	0.049	0.777	0.313	0.313	6/6 ($p = 0.031$)
PLR cells	0.118	0.683	0.197	0.197	5/6 ($p = 0.22$)
ALP cells	0.140	0.823	0.477	0.477	3/6 ($p = 1.0$)
Global annealing	0.003	0.843	0.000	0.000	6/6 ($p = 0.031$)
Anneal + balanced replay‡	0.006	0.803	0.000	0.000	6/6 ($p = 0.031$)
Asynchronous global mixture	0.125	0.757	0.383	0.383	3/6 ($p = 1.0$)
Random scaffold	0.117	0.803	0.477	0.477	4/6 ($p = 0.69$)
Fixed mid scaffold	0.022	0.753	0.053	0.053	5/6 ($p = 0.22$)
Target start	0.000	0.880	0.000	0.000	6/6 ($p = 0.031$)

Table A11: Horizon and ladder-resolution robustness of the push $\times$ shelf-place conflict (cells: worst-group AUC / shelf-place final). Annealing continues to fail with a longer horizon at five levels, but succeeds with nine levels.

Protocol (ladder, budget, seeds)	SCOUT	Global annealing	Global mixture
5 levels, 6000 it, 6 seeds	0.141 / 0.617	0.003 / 0.000	0.125 / 0.383
5 levels, 9000 it, 3 seeds	0.337 / 0.680	0.008 / 0.000	0.347 / 0.853
9 levels, 9000 it, 2 seeds	0.367 / 0.840	0.353 / 0.890	0.374 / 0.810

as a curriculum solves both groups, with final success of 1.00 on the easy group and 0.92 on the hard group. The benefit comes from moving across levels rather than selecting one level. Constructed DoorKey-16 $\times$ 16 solution-length splits identify a boundary for the mixture baseline. SCOUT outperforms global annealing on the worst group on all three seeds in two bank compositions, with about 2 $\times$ the worst-group AUC. However, the asynchronous mixture ties SCOUT. This result is consistent with cross-level transfer. MiniGrid restore states lie on the solution path of the target-start task, and the convolutional policy generalizes across levels. Population-level mixed exposure can therefore train $\ell=0$ directly.

Scope. Together with the contrastive learner result (main paper, Results), the DoorKey splits identify two cases in which SCOUT provides less benefit. A strong directed learner can remove the *access* problem, and strong cross-level generalization can substitute for *allocation*. The conflict setting instead requires chain-dependent tasks with weak cross-level transfer.

D Ablations

Hyperparameter screens. Tables A12 and A13 report the development-seed screens behind the frozen controller constants (selection and freezing protocol in Appendix A.1). In the antistall screen, every enabled stale limit beats the disabled controller on worst-group AUC and raises the hard group’s final success to at least 0.92. Thus, antistall helps on these development seeds. Differences among $N_{\text{stale}} \in \{2, 4, 8\}$ are not monotonic and are small relative to the variation across three seeds. The annealing length has little effect on PointMaze Large. Every tested length gives a similarly low development-seed AUC.

Critic uncertainty ablation. Adding a critic uncertainty term to the sampling or transition signal did not consistently improve performance. It could also delay progression near sparse-reward discontinuities, so we exclude it. The code release will include full per-seed tables and the conflict confirmation block.

Table A12: Antistall sweep (conflict, three development seeds). Worst-group AUC is the median over seeds; final success columns are medians by group.

N_{stale}	worst-group AUC	on-table final	hard final
0 (off)	0.15	0.96	0.84
2	0.49	1.00	1.00
4	0.20	1.00	0.92
8	0.39	1.00	0.96

Table A13: PointMaze Large annealing-length screen (development seeds 40–42). Median held-out AUC and final success by schedule length T_{anneal} .

T_{anneal}	AUC	final
1000	0.00	0.00
1500	0.05	0.00
2500	0.04	0.00
3500	0.03	0.00
5000	0.05	0.03

Evidence-gated transition rules. To test whether the EMA-threshold transition is one possible stable local controller, we test two evidence-gated variants. They differ from SCOUT only in the transition rule. *TP-SCOUT* uses Wilson bounds over discounted success counts, while *B-SCOUT* uses a discounted Beta posterior. Fig. A5 shows a six-seed replication on the automatic conflict. All stable local-frontier rules perform similarly and outperform both global schedules. Evidence gating reduces frontier reversals by about half at the same worst-group AUC. On the coarser hand-built ladder, the gated rules perform somewhat better. The success statistics must discount old evidence. An undiscounted posterior ($\gamma=1$) reduces worst-group AUC to 0.58 because it cannot adapt to a changing policy. We also tested a drift-aware Bayesian gate across 23 tuned configurations. Its best configuration uses no anticipation and still performs worse than every simple rule. These results do not distinguish among the stable local rules. They indicate that the shared local frontier, rather than one specific transition rule, provides the main benefit (main paper, Results).

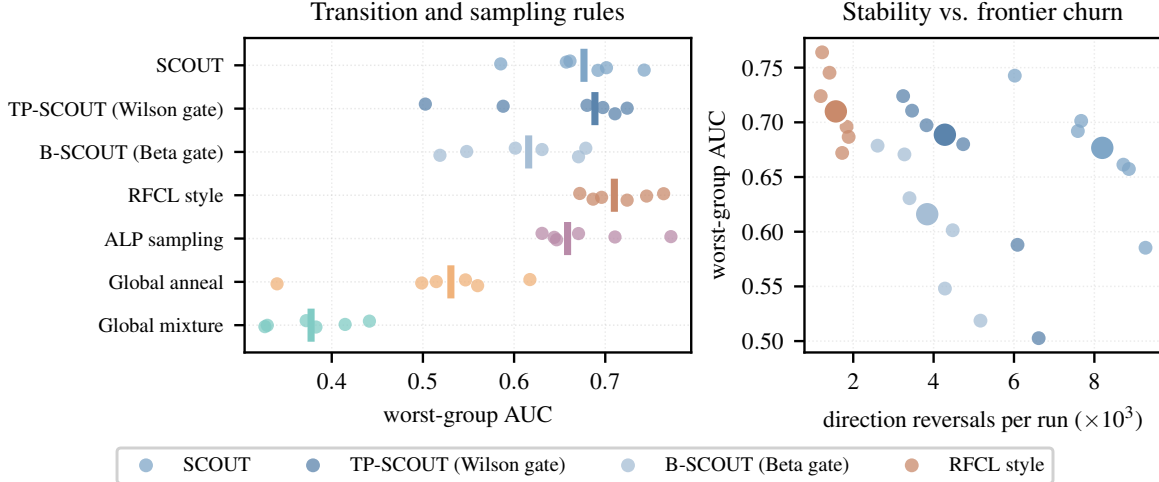


Figure A5: Transition and sampling rules on the automatic constructed conflict (six seeds; this block’s common window). **Left:** worst-group AUC per rule (per-seed dots, median ticks): the stable local-frontier rules outperform both global schedules, with no significant pairwise differences among them. **Right:** per-context direction reversals against worst-group AUC (large markers medians): evidence gating halves reversals at equal worst-group AUC; the advance-biased RFCL-style rule churns least.